

Vite & Gourmand

Documentation technique

ECF TP Développeur Web et Web Mobile, Studi

Élément	Valeur
Auteur	Arthur Gatoux, FastDev
Dépôt	github.com/arthurgatoux/vite-gourmand
Production	https://vite-gourmand-ten.vercel.app/
Version	1.0, 20 septembre 2026

1. Finalité et architecture générale

Vite & Gourmand est une application web de commande de menus événementiels. Elle distingue les visiteurs, les utilisateurs inscrits, les employés et les administrateurs. L'architecture retenue sépare les données transactionnelles des statistiques, sans séparer artificiellement le front et le back dans deux projets indépendants.

Next.js assure l'interface, le routage et la logique serveur. Supabase fournit PostgreSQL et l'authentification. MongoDB Atlas porte uniquement les agrégats nécessaires au tableau de bord administrateur. Vercel héberge l'application et déclenche les déploiements depuis GitHub.

Brique	Responsabilité	Justification
Next.js, App Router, TypeScript	Interface, routes, Server Components et Server Actions	Un seul projet pour le front et le back ; logique métier sensible exécutée côté serveur.
Supabase	PostgreSQL, Auth, Storage, RLS	Données relationnelles, authentification et contrôle d'accès directement au niveau base.
MongoDB Atlas	Statistiques agrégées	Les données de commandes par menu et de chiffre d'affaires se prêtent à un document agrégé.
Resend	Emails transactionnels	Bienvenue, confirmation de commande, avis et retour de matériel.
Vercel	Déploiement et production	Intégration naturelle avec Next.js et publication depuis la branche main.
Notion	Suivi et documentation	Backlog, décisions et documents de projet accessibles au jury.

Le choix d'une base relationnelle et d'une base non relationnelle répond à une exigence explicite du sujet. MongoDB ne remplace pas PostgreSQL : il évite de faire porter les statistiques du dashboard sur les tables transactionnelles.

2. Configuration de l'environnement

Le projet a été initialisé depuis le template Next.js with-supabase. Le développement local utilise Node.js, npm et un fichier .env.local exclu du versionnement. Les secrets ne sont donc jamais déposés dans le dépôt public.

Variable	Utilisation
NEXT_PUBLIC_SUPABASE_URL	URL publique du projet Supabase.
NEXT_PUBLIC_SUPABASE_PUBLIC_KEY	Clé publique utilisée par les clients Supabase navigateur et serveur.
SUPABASE_SERVICE_ROLE_KEY	Accès serveur privilégié, réservé aux opérations d'administration.
MONGODB_URI	Connexion serveur à MongoDB Atlas pour les statistiques.

Commandes locales principales

```
npm install
npm run dev
npm run lint
npm run build
```

La commande npm run build est particulièrement importante : contrairement au serveur de développement, elle exécute le contrôle TypeScript complet utilisé par Vercel avant publication.

3. Organisation du code

Dossier	Contenu
app	Routes Next.js App Router, pages publiques, espaces utilisateur, employé et administrateur.
components	Composants réutilisables d'interface et composants métier.
lib/supabase	Clients Supabase, requêtes, Server Actions, règles de rôle et helpers.
lib/mongodb	Connexion MongoDB et requêtes du dashboard.
supabase/sql	Scripts SQL de création de schéma et d'intégration des données de test.
supabase/migrations	Évolutions de schéma, RLS, triggers et correctifs de sécurité.
docs	MCD, diagrammes, RGPD, charte, tests et documentation associée.
mongodb/init	Script d'initialisation de la collection stats_menus.

4. Modèle relationnel PostgreSQL

Le modèle relationnel s'organise autour du profil, du catalogue et de la commande. Le schéma complet est disponible dans le dépôt à l'emplacement docs/MCD-ViteGourmand.md et dans le script supabase/sql/001_create_schema.sql.

Domaine	Tables principales	Rôle
Comptes	profils	Informations métier du compte Supabase : identité, contact, rôle et état du compte.
Catalogue	menus, menu_images, plats, allergenes, regimes	Description des menus, galerie, plats, allergènes et régimes.
Associations	menu_plat, plat_allergene, menu_regime	Relations plusieurs à plusieurs du catalogue.
Commandes	commandes, historique_statut_commande, prets_materiel	Prestation, calculs, suivi horodaté et restitution du matériel.
Relation client	avis, messages_contact	Modération des avis et demandes envoyées par les visiteurs.

Les types énumérés sécurisent les valeurs métier importantes : rôle utilisateur, type de plat, statut d'avis et statut de commande. Les tables sensibles sont protégées par Row Level Security.

Règles métier portées par la base

Règle	Mécanisme
Prix de commande	Trigger calculer_prix_commande : minimum de personnes, réduction de 10 %, livraison et total recalculés côté base.
Menu non modifiable	Trigger empecher_changement_menu_commande à la modification d'une commande.
Historique fiable	Trigger après changement de statut : une nouvelle ligne horodatée est créée, sans réécrire l'historique.
Profil à l'inscription	Trigger handle_new_user : création du profil avec rôle utilisateur par défaut.
Rôle protégé	Trigger prevent_role_escalation : un utilisateur ne peut jamais s'attribuer un rôle supérieur.

5. Base NoSQL MongoDB

MongoDB Atlas héberge la base vite_gourmand_stats et la collection stats_menus. Chaque document regroupe les indicateurs d'un menu : identifiant PostgreSQL, titre, thème, nombre total de commandes, chiffre d'affaires total, historique mensuel et date de mise à jour.

```
stats_menus
  menu_id: string
  titre_menu: string
  theme: string
  nb_commandes_total: int
  chiffre_affaires_total: double
  historique_mensuel: [{ mois, nb_commandes, chiffre_affaires }]
  derniere_maj: date
```

La synchronisation se déclenche lorsqu'une commande passe au statut terminé. Elle est exécutée côté serveur afin de ne pas exposer la chaîne MongoDB au navigateur. Le dashboard administrateur lit ensuite MongoDB pour comparer les menus et filtrer le chiffre d'affaires par menu ou période.

6. Authentification, rôles et sécurité

Supabase Auth gère le hachage des mots de passe et les sessions. L'application complète cette protection avec des contrôles de rôle dans les layouts et les Server Actions. La RLS reste néanmoins le verrou principal : même une requête directe à l'API Supabase ne doit pas permettre de lire ou modifier une donnée interdite.

Rôle	Accès accordé
Visiteur	Catalogue, filtres, pages légales, contact et inscription.
Utilisateur	Ses données personnelles, ses commandes, leur suivi et ses avis.
Employé	Gestion opérationnelle des commandes, catalogue, horaires et modération des avis.
Administrateur	Toutes les capacités employé, gestion des comptes employés et statistiques MongoDB.

Défense en profondeur

Niveau	Disposition
Interface	Routes et navigation adaptées au rôle connecté.
Server Actions	Relecture du profil courant, validation des données et contrôle des transitions autorisées.
Base PostgreSQL	RLS sur les tables ; politiques basées sur <code>auth.uid()</code> et la fonction <code>est_employe_ou_admin()</code> .
Fonctions sensibles	Fonctions SECURITY DEFINER contrôlant explicitement le rôle avant toute écriture.
Secrets	Clés d'API dans les variables d'environnement ou Supabase Vault, jamais dans le client.

Un audit RLS a corrigé une faille sur l'insertion des commandes. La policy vérifie désormais que `utilisateur_id` est égal à `auth.uid()`, ce qui bloque l'usurpation d'une commande au nom d'un autre compte.

7. Logique serveur et emails

Les mutations utilisateur importantes sont réalisées via des Server Actions : création, modification et annulation de commande, modification du profil et dépôt d'avis. Ce choix évite de s'appuyer seulement sur la validation du navigateur, qui peut être contournée.

Événement	Traitement automatique
Création de compte	Création du profil, rôle utilisateur par défaut et email de bienvenue.
Création de commande	Calcul des montants, décrémentation du stock, historique initial et email de confirmation.
Commande terminée	Email invitant le client à déposer un avis.
Prêt de matériel	Calcul de la date limite à dix jours ouvrés et notification au client.
Statut terminé	Synchronisation des données agrégées vers MongoDB.

Les emails passent par `pg_net` vers l'API Resend. La clé Resend est stockée dans Supabase Vault. Le domaine `mail.gatouxweb.com` a été vérifié avant la mise en production afin que les messages puissent être délivrés à de vrais destinataires.

8. Accessibilité et protection des données

La charte graphique utilise les couleurs bordeaux, or, crème et charbon avec des contrastes contrôlés. L'interface possède un lien d'évitement vers le contenu, un focus clavier visible et des messages d'erreur reliés aux champs concernés via `aria-describedby`. Les formulaires sont conçus pour être utilisables au clavier et sur mobile.

Les données collectées restent limitées au besoin de commande : identité, email, téléphone et adresses. Supabase est hébergé en région européenne. MongoDB ne contient que des statistiques agrégées et aucune donnée personnelle du client. Les durées de conservation, droits d'accès, rectification, effacement et portabilité sont documentés dans docs/RGPD-ViteGourmand.md.

9. Diagrammes de conception

Le dépôt contient les livrables de conception attendus, directement consultables dans le dossier docs. Ils complètent cette documentation sans répéter des pages de diagrammes sous forme d'images peu lisibles.

Document	Contenu
docs/MCD-ViteGourmand.md	Modèle conceptuel relationnel, dictionnaire de données et règles de gestion.
docs/NoSQL-MongoDB-ViteGourmand.md	Schéma de la collection stats_menus, index et requêtes du dashboard.
docs/Diagrammes-Utilisation-ViteGourmand.md	Cas d'utilisation pour visiteur, utilisateur, employé et administrateur.
docs/Diagrammes-Sequence-ViteGourmand.md	Séquences inscription, commande, changement de statut et consultation du dashboard.
docs/RGPD-ViteGourmand.md	Registre simplifié des traitements, minimisation, durées et sous traitants.

Le choix de conserver ces diagrammes au format Mermaid dans le dépôt permet de les versionner avec le code et de maintenir leur lisibilité, plutôt que de figer des captures devenues rapidement obsolètes.

10. Déploiement et recette de production

L'application est déployée sur Vercel. Les variables nécessaires à Supabase et MongoDB sont configurées dans l'environnement de production. Un merge vers main déclenche le pipeline de déploiement, puis une vérification fonctionnelle est menée directement sur l'URL publique.

```
vercel login
vercel link
vercel env add NOM_VARIABLE production
vercel --prod
```

Trois ajustements ont été nécessaires pour obtenir un build de production stable : correction du typage du formulaire de contact, typage explicite de la collection MongoDB et retrait de l'option cacheComponents incompatible avec les routes qui utilisent les cookies de session. Ces corrections n'ont pas changé les règles métier.

11. Tests et maintenance

La recette comporte 41 cas de test répartis par rôle. Elle couvre les parcours de connexion, le catalogue et ses filtres, la commande, le suivi, les actions employé, le dashboard administrateur, la séparation des droits, les formulaires RGAA et l'affichage mobile.

Le projet reste maintenable parce que les règles critiques sont localisées. Les contrôles de rôle sont dans les helpers, layouts et Server Actions ; les règles de persistance dans les migrations et triggers ; les composants d'interface dans leurs dossiers métier. Lorsqu'une règle évolue, il est donc possible de savoir rapidement où intervenir.